МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
Львівський національний університет імені Івана Франка
Факультет електроніки та комп’ютерних технологій

# ЗВІТ

з лабораторної роботи №8

з дисципліни “Професійний Python”

на тему “Мережеве та асинхронне програмування. REST API”

Виконала
студентка 3 курсу
групи ФеП-33с
Корхова Марія Сергіївна

Перевірив
професор
Свелеба Сергій Андрійович

Львів 2026

Мета роботи: Ознайомитися з основами мережевого програмування у Python, принципами роботи протоколу HTTP, архітектурним стилем REST та форматом JSON.

Варіант 7. Система оцінювання студентів Ресурси: students, subjects, grades. Endpoints: POST /students; POST /subjects; POST /grades; GET /students/{id}/grades; GET /students/{id}/average; PATCH /grades/{id}; DELETE /grades/{id}; GET /students?min_average={value}. Реалізувати: створення студента і дисципліни; виставлення/редагування/видалення оцінки; середній, мінімальний і максимальний бал; визначення успішності; сортування студентів; пошук студентів із середнім балом вище заданого. Асинхронне завдання: паралельно отримувати академічну статистику декількох студентів із Semaphore, timeout і retry.

Структура проєкту

```
Professional_Python_Lab_8_Variant_7_Grades/
├── pyproject.toml
├── README.md
├── src/grade_manager/
│   ├── __init__.py
│   ├── main.py
│   ├── api.py
│   ├── schemas.py
│   ├── database.py
│   ├── models.py
│   ├── repositories.py
│   ├── services.py
│   └── external_api.py
├── tests/
│   ├── conftest.py
│   ├── test_api.py
│   └── test_external_api.py
└── report/
    └── Звіт_ЛР8_Варіант_7.md
```

Опис REST API

HTTP methodEndpointПризначенняОчікуваний статус**GET**`/students`Отримання списку студентів з можливістю фільтрації200**POST**`/students`Створення нового студента201/422**POST**`/subjects`Створення нової дисципліни201/422**POST**`/grades`Виставлення оцінки студенту з предмета201/422**GET**`/students/{id}/grades`Отримання списку оцінок конкретного студента200/404**GET**`/students/{id}/average`Отримання середнього бала студента200/404**PATCH**`/grades/{id}`Редагування існуючої оцінки200/404/422**DELETE**`/grades/{id}`Видалення оцінки204/404**POST**`/students/batch-average`Паралельне отримання середніх балів кількох студентів200/504

5. Path та query parameters

Приклад path parameter: GET /students/1, де 1 — ідентифікатор студента.

Приклад query parameters: GET /students?min_average=90.0. Параметри дозволяють фільтрувати результати (наприклад, студентів із середнім балом вище заданого).

6. Pydantic schemas і правила валідації

StudentCreate перевіряє ім'я та прізвище (від 2 до 100 символів). GradeCreate вимагає додатні ID для студента й предмета та перевіряє значення оцінки (від 0 до 100). Некоректні дані відхиляються зі статусом 422.

7. JSON request і response

Приклад POST /grades:

```
{
  "student_id": 1,
  "subject_id": 1,
  "value": 95.5
}
```

Приклад відповіді (ID формується базою даних):

```
{
```

```
  "id": 1,
  "value": 95.5,
  "student_id": 1,
  "subject_id": 1
}
```

Асинхронна частина

external_api.py використовує httpx.AsyncClient для HTTP GET, async/await для очікування відповіді, asyncio.create_task() для створення задач, asyncio.gather() для паралельного виконання та asyncio.wait_for() для timeout. asyncio.Semaphore обмежує кількість одночасних HTTP-запитів. При тимчасових помилках повторюються запити з експоненційною затримкою. Кінцева точка POST /students/batch-average запускає агрегування для кількох студентів.

9. Persistence layer та архітектура

Використано SQLAlchemy та SQLite. Моделі Student, Subject та Grade зберігаються в базі даних. Репозиторії відповідають за запити до БД, сервісний рівень — за бізнес-логіку та обробку конфліктів, API-рівень — за HTTP і статус-коди. Це відповідає ланцюжку HTTP client → FastAPI → Pydantic → Service → Repository → SQLite.

10. Автоматизоване тестування

Тести перевіряють створення та читання даних, оновлення, видалення, валідацію, фільтрацію, статистику, повторні спроби HTTP-запиту й обмеження паралельності. Запуск: python -m pytest. Фактичний результат потрібно отримати у власному середовищі та зафіксувати скриншотом.

11. Результат запуску тестів

```
korkhovamariia@MariiaKorkhova:~/prof_Python/prof_Python/prof_Python$ code Lab_08
korkhovamariia@MariiaKorkhova:~/prof_Python/prof_Python/prof_Python$ cd Lab_08
korkhovamariia@MariiaKorkhova:~/prof_Python/prof_Python/prof_Python/Lab_08$ python3 -m venv .venv
korkhovamariia@MariiaKorkhova:~/prof_Python/prof_Python/prof_Python/Lab_08$ source .venv/bin/activate
```

## 12. Відповіді на контрольні запитання

1. Що таке комп'ютерна мережа? Сукупність пристроїв, які обмінюються даними за спільними правилами.

2. Що таке клієнт-серверна архітектура? Модель, у якій клієнт надсилає запит, а сервер його обробляє та повертає відповідь.
3. Що таке HTTP? Протокол передавання даних між клієнтом і сервером.
4. Що містить HTTP request? Метод, URL, заголовки та, за потреби, тіло.
5. Що містить HTTP response? Код статусу, заголовки та, за потреби, тіло відповіді.
6. Для чого GET? Для отримання ресурсу.
7. Для чого POST? Для створення ресурсу або запуску операції.
8. Для чого PUT? Для повного оновлення ресурсу.
9. Для чого PATCH? Для часткового оновлення ресурсу.
10. Для чого DELETE? Для видалення ресурсу.
11. Що означає 200? Запит успішно виконано.
12. Що означає 201? Ресурс створено.
13. Що означає 204? Запит успішний, тіло відповіді відсутнє.
14. Що означає 404? Ресурс не знайдено.
15. Що означає 422? Дані запиту не пройшли валідацію.
16. Що таке JSON? Текстовий формат подання структурованих даних.
17. Що таке REST? Архітектурний стиль побудови мережевих сервісів навколо ресурсів.
18. Що таке resource у REST? Сутність, доступна за URL, наприклад книга або автор.
19. Що таке endpoint? Конкретний шлях API разом із HTTP-методом.
20. Що таке path parameter? Значення в самому шляху URL, наприклад /books/5.
21. Що таке query parameter? Параметр після ? у URL, наприклад ?page=2.
22. Що таке request body? Дані, які клієнт передає в тілі запиту.
23. Що таке FastAPI? Python-фреймворк для створення API.
24. Для чого Uvicorn? Для запуску ASGI-застосунків.
25. Для чого Swagger UI? Для перегляду документації API та ручного виконання запитів.
26. Що таке Pydantic? Бібліотека для опису схем, перевірки та перетворення даних.
27. Що таке validation? Перевірка відповідності даних заданим правилам.
28. Що таке response model? Схема, яка визначає структуру відповіді API.
29. Що таке synchronous programming? Виконання операцій послідовно з очікуванням завершення кожної.
30. Що таке asynchronous programming? Підхід, який дозволяє перемикатися між операціями під час очікування.
31. Для чого async? Для оголошення coroutine.
32. Для чого await? Для асинхронного очікування результату coroutine або awaitable.
33. Що таке coroutine? Асинхронна функція/обчислення, виконанням якого керує event loop.

34. Що таке event loop? Цикл, який планує та виконує асинхронні задачі.
35. Що таке Task? Об'єкт, який планує coroutine для виконання в event loop.
36. Для чого asyncio.create_task()? Щоб запланувати coroutine як окрему задачу.
37. Для чого asyncio.gather()? Щоб очікувати завершення кількох awaitable-операцій.
38. Що таке timeout? Обмеження часу, після якого операція припиняється або завершується помилкою.
39. Для чого retry? Для повторення невдалої операції, якщо помилка може бути тимчасовою.
40. Що таке backoff? Затримка між повторними спробами, часто зі збільшенням після кожної помилки.
41. Для чого Semaphore? Для обмеження кількості одночасних операцій.
42. Що таке HTTPX? HTTP-клієнт Python із синхронним та асинхронним API.
43. Чим AsyncClient відрізняється від синхронного клієнта? Він виконує HTTP-запити асинхронно й не блокує event loop під час очікування мережі.
44. Чому AsyncClient бажано повторно використовувати? Для повторного використання пулу з'єднань і зменшення накладних витрат.
45. Навіщо обмежувати паралелізм? Щоб не перевантажувати сервіс, мережу або локальні ресурси.
46. Чому не запускати тисячі tasks одночасно? Це може витратити надмірні ресурси та перевантажити зовнішній сервіс.
47. Як REST API взаємодіє з Repository Layer? Endpoint викликає сервіс, який використовує репозиторій для читання або зміни даних.
48. Як FastAPI працює з Pydantic? Перевіряє request body, формує схеми документації та серіалізує response models.
49. Як протестувати REST API? Надсилати тестові HTTP-запити за допомогою FastAPI TestClient або HTTPX і перевіряти статуси та дані.
50. Чим API integration test відрізняється від unit test? Integration test перевіряє взаємодію кількох компонентів, а unit test — окрему одиницю ізольовано.

## Висновок

У лабораторній роботі було розроблено REST API для системи оцінювання студентів. Реалізовано основні HTTP-операції, перевірку даних через Pydantic, збереження в SQLite через SQLAlchemy, фільтрацію та обчислення статистики. Для асинхронної взаємодії використано HTTPX, async/await, Task, gather, timeout, retry та Semaphore. Підготовлено автоматизовані тести, які дають змогу перевірити роботу API.